```
$card = new Card(Suit::SPADES, 'ace');
print "The {$card->getFullName()}, the {$card->getFullName()} \m/" . PHP_EOL;
```

```
Output:
Fatal error: Uncaught Error: Object of class Suit could not be converted to
string...
```

In their basic (sometimes called "pure") form, enum cases don't actually have values, or rather they don't have scalar values. Pure enums cannot be cast to strings or any other scalar type, which breaks `getFullName()` and `getSuit()`, which have `string` type declarations and perform string operations. When we implemented `Suit::SPADES` as a string constant, we were able to freely use it in a string expression. That is not the case when we use a pure enum, but fortunately there's a solution.

Backed Enums

A backed enum gives each enumerated case a scalar value, with the following syntax:

```
enum Suit: string {
    case CLUBS = 'clubs';
    case DIAMONDS = 'diamonds';
    case HEARTS = 'hearts';
    case SPADES = 'spades';
}
```

By giving the enum a scalar type with `enum Suit: string`, we are declaring that the enum is backed by a scalar type of string. Naturally this means that all values for each case must be strings, and additionally all cases must have a value. A backed enum is not permitted to have any cases without values.

To access the enum value a little extra code is required due to internal implementation details. Each case is an object in its own right, and we have to access a read-only property to get to the value set in the declaration:

```
    public function getFullName(): string
    {
        return "{$this->value} of {$this->suit->value}";
    }
```